2.2.7 Shell Arithmetic using `expr` and `bc` Commands

In shell scripting, arithmetic operations can be performed using different commands:

- **`expr` Command**: Used for integer arithmetic operations within shell scripts. It evaluates and prints the result of expressions.

  Example usage:

  ```
  result=$(expr 5 + 3)   # Adds 5 and 3
  echo "Result: $result" # Output: Result: 8
  ```

  Supported operators in `expr` include addition (+), subtraction (-), multiplication (*), division (/), modulus (

- **`bc` Command**: Stands for 'Basic Calculator' and supports floating-point arithmetic and advanced mathematical functions.

  Example usage:

  ```
  result=$(echo "5.5 + 3.2" | bc) # Adds 5.5 and 3.2
  echo "Result: $result"          # Output: Result: 8.7
  ```

  `bc` handles floating-point arithmetic and provides functions like sine, cosine, square root, etc., for more complex calculations.

These commands offer basic arithmetic functionalities within shell scripts. `expr` is suitable for simple integer arithmetic, while `bc` provides a broader range of mathematical operations and supports floating-point numbers.

2.2.8 Flow Control in Shell Scripting using `if` Statements

Shell scripting offers various forms of `if` statements for conditional flow control:

1. **Basic `if` Statement:**

   ```
   if [ condition ]; then
       # Commands to execute if the condition is true
   fi
   ```

2. **`if-else` Statement:**

   ```
   if [ condition ]; then
       # Commands to execute if the condition is true
   else
       # Commands to execute if the condition is false
   fi
   ```

3. **if-elif-else Statement:**

```
if [ condition1 ]; then
    # Commands to execute if condition1 is true
elif [ condition2 ]; then
    # Commands to execute if condition2 is true
else
    # Commands to execute if both condition1 and condition2 are false
fi
```

4. **Nested if Statements:**

```
if [ condition1 ]; then
    if [ condition2 ]; then
        # Commands to execute if both condition1 and condition2 are true
    fi
fi
```

5. **if Statement with Logical Operators:**

```
if [ condition1 -a condition2 ]; then
    # Commands to execute if condition1 AND condition2 are true
fi

if [ condition1 -o condition2 ]; then
    # Commands to execute if condition1 OR condition2 is true
fi
```

These variations enable conditional execution of commands based on different conditions within shell scripts, offering flexibility in controlling the flow of the script.

2.2.9 Operators in Shell Scripting

In shell scripting, operators are used to perform various operations on variables, constants, and expressions. Here are the common types of operators:

1. Arithmetic Operators

- `+`, `-`, `*`, `/`, `%`: Perform addition, subtraction, multiplication, division, and modulus respectively.

2. Relational Operators

- `-eq`, `-ne`, `-gt`, `-lt`, `-ge`, `-le`: Compare numbers (equal, not equal, greater than, less than, greater than or equal to, less than or equal to) within `test` or `[ ]` brackets.

Example:

```
fruit="apple"

case $fruit in
    apple)
        echo "It's an apple."
        ;;
    banana|orange)
        echo "It's a banana or an orange."
        ;;
    *)
        echo "It's another fruit."
        ;;
esac
```

This example evaluates the variable `$fruit` against different patterns and executes respective blocks based on the matching pattern.

The `case` structure simplifies code readability when multiple conditions need evaluation against a single variable or expression.

2.2.11 Loops in Shell Scripting

In shell scripting, loops are used to execute a block of code repeatedly based on certain conditions. There are various types of loops available:

1. `for` Loop

The `for` loop iterates through a list of items or values. It is suitable when you have a known set of elements to loop through.

Syntax:

```
for variable in list
do
    # Commands to execute for each iteration
done
```

Example:

```
for i in 1 2 3 4 5
do
    echo "Iteration: $i"
done
```

2. `while` Loop

The `while` loop executes a block of code as long as a specified condition remains true.

Syntax:

```
while [ condition ]
do
    # Commands to execute as long as the condition is true
done
```

Example:

```
count=1
while [ $count -le 5 ]
do
    echo "Count: $count"
    ((count++))
done
```

3. until Loop

The `until` loop executes a block of code until a specified condition becomes true.

Syntax:

```
until [ condition ]
do
    # Commands to execute until the condition becomes true
done
```

Example:

```
count=1
until [ $count -gt 5 ]
do
    echo "Count: $count"
    ((count++))
done
```

4. Nested Loops

You can nest loops within each other to create more complex control structures.

Example:

```
for i in {1..3}
do
    echo "Outer Loop Iteration: $i"
    for j in A B C
    do
        echo "   Inner Loop Iteration: $j"
    done
done
```

These loops in shell scripting provide various ways to iterate through data, execute code repeatedly based on conditions, and control the flow of a script.

6. **Exit Code Analysis (Optional):**

- After script execution, check the exit code by typing:

```
echo $?
```

An exit code of `0` generally indicates success, while other codes signify different types of errors or warnings.

Video Reference:

https://youtube.com/playlist?list=PLWjmN065fOfGdAZrlP6316HVIIh8jlve

2.3.1 Lab Exercises

Exercise 1: Write a shell script that calculates the factorial of a user-provided number using a `for` loop.

Exercise 2: Develop a script that prints the multiplication table for a given number up to 10 using a `for` loop.

Exercise 3: Create a script that reverses a given number using a `while` loop.

Exercise 4: Write a script to display all files in a directory using a `for` loop.

Exercise 5: Write a shell script named `day_of_week.sh` that prompts the user to enter a number between 1 and 7, representing a day of the week. Use a `case` structure to print the corresponding day of the week:

- If the user enters `1`, print "Sunday".
- If the user enters `2`, print "Monday".
- If the user enters `3`, print "Tuesday".
- If the user enters `4`, print "Wednesday".
- If the user enters `5`, print "Thursday".
- If the user enters `6`, print "Friday".
- If the user enters `7`, print "Saturday".
- If the user enters a number outside this range, print "Invalid input. Please enter a number between 1 and 7".

Exercise 6: Create a menu-driven calculator script that performs basic arithmetic operations based on user selection using a `case` structure.

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fileDescriptor;

    // Create a new file named "file.txt" and open it for reading
    fileDescriptor = open("file.txt", O_CREAT | O_RDONLY, 0644);

    close(fileDescriptor); // Close the file

    return 0;
}
```

2. A C program to Read from Console and Write to Console

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main() {
    char buffer[BUFFER_SIZE];
    //0 is the file descriptor of standard input.
    ssize_t bytesRead = read(0, buffer, BUFFER_SIZE);
    write(1, buffer, bytesRead); //1 is the file descriptor of standard output.

    return 0;
}
```

3. A C program to Append Data into a File

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
    char data[] = "This data will be appended to the file.\n";
    int fileDescriptor;

    fileDescriptor = open("file.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
    write(fileDescriptor, data, strlen(data));
```

```
    close(fileDescriptor);

    return 0;
}
```

4. A C program to Read from and Write to Files

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main() {
    char buffer[BUFFER_SIZE];
    int readFd, writeFd;

    readFd = open("source.txt", O_RDONLY);
    writeFd = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    ssize_t bytesRead;
    while ((bytesRead = read(readFd, buffer, BUFFER_SIZE)) > 0) {
        write(writeFd, buffer, bytesRead);
    }

    close(readFd);
    close(writeFd);

    return 0;
}
```

5. A C program that reads characters from the 11th to the 20th position from a file named "input.txt" using the lseek system call.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 11

int main() {
    int fileDescriptor;
    char buffer[BUFFER_SIZE];

    fileDescriptor = open("input.txt", O_RDONLY);
```

```
    // Attempt to rename the file
    if (rename(old_name, new_name) != 0) {
        // If rename fails, print an error message
        printf("Error renaming the file");
        return -1;
    }

    // Confirm successful renaming
    printf("File renamed from '%s' to '%s'.\n", old_name, new_name);

    return 0;
}
```

8. C Program to Demonstrate the Use of stat System Call

```
#include <stdio.h>
#include <sys/stat.h>  // For struct stat and stat()
#include <stdlib.h>    // For exit()
#include <errno.h>     // For errno
#include <string.h>    // For strerror()

int main() {
    // Name of the file to retrieve information about
    const char *filename = "sample_file.txt";

    // Create a struct stat to hold file information
    struct stat fileStat;

    // Retrieve file status information
    if (stat(filename, &fileStat) != 0) {
        // If stat fails, print an error message
        printf("Error retrieving file information");
        return -1;
    }

    // Print file status information
    printf("File: %s\n", filename);
    printf("File Size: %ld bytes\n", fileStat.st_size);
    printf("Number of Links: %ld\n", fileStat.st_nlink);
    return 0;
}
```

Video Reference:

https://youtube.com/playlist?list=PLWjmN065fOfGdAZrlP6316HVHh8jlve

3.3 Lab Exercises

Exercise 1: Write a program in C using system calls that lets users choose to copy either the first half or the second half of a file by entering 1 or 2.

Exercise 2: Create a C program using system calls that keeps reading from the console until the user types '$'. Save the input data to a file called 'input.txt'."

Exercise 3: Write a C program that encrypts a text file using a simple encryption technique and saves the encrypted content to a new file.
Requirements:
Input: Provide a text file named "input.txt" with plain text content.
Encryption Technique: Shift each character in the file content by a fixed number of positions (e.g., shifting each character by 3 positions in the ASCII table).
Output: Save the encrypted content to a new file named "encrypted.txt".

Exercise 4: Write a C program to the following tasks:

(a) Create a new file named `sample.txt` in your current directory.

(b) Using chmod system call to change the permissions of `sample.txt` so that only the owner has read and write permissions, while the group and others have no permissions. Verify the changes using the `ls -l` command.

(c) Change the ownership of `sample.txt` to a different user (replace `username` with the actual username of the different user) using the `chown` command. Verify the changes using the `ls -l` command.

(d) Create a hard link named `sample_link.txt` to `sample.txt` using the `link` system call. Verify the existence and properties of the hard link using the `ls -li` command.

(e) Create a symbolic link named `sample_symlink.txt` to `sample.txt` using the `symlink` system call. Verify the existence and properties of the symbolic link using the `ls -l` command.

3.4 Sample Viva Questions

Question 1: What is the purpose of the open system call?

Question 2: Discuss the parameters of the read system call.

- `unsigned short int d_reclen`: It denotes the length of this record.
- `unsigned char d_type`: This member identifies the type of the file. For example, `DT_DIR` for directories, `DT_REG` for regular files, and others based on the file type.
- `char d_name[]`: This member holds the name of the directory entry. It is a character array representing the name of the file or directory.

4.3 Sample Programs

1. A C program that prints the contents of a directory using system calls like opendir, readdir, and closedir

```
#include <stdio.h>
#include <dirent.h>

int main() {
    DIR *dir;
    struct dirent *entry;

    dir = opendir(".");

    if (dir) {
        printf("Contents of the directory:\n");
        while ((entry = readdir(dir)) != NULL) {
            printf("%s\n", entry->d_name);
        }
        closedir(dir);
    }

    return 0;
}
```

2. Write a C program to create a new directory named "NewDirectory" within the file system

```
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>

int main() {
    const char *dirname = "NewDirectory";

    // Creating a new directory named "NewDirectory"
    mkdir(dirname, 0777);
```

```
    return 0;
}
```

3. C Program to Demonstrate the Use of getcwd, chdir, and rmdir System Calls

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>  // For getcwd(), chdir(), and rmdir()
#include <errno.h>   // For errno
#include <string.h>  // For strerror()
#include <sys/types.h> // For mkdir()
#include <sys/stat.h>  // For mkdir()

#define NEW_DIR "new_directory"

int main() {
    char cwd[1024];  // Buffer to store current working directory
    char *original_dir;

    // Get the current working directory
    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        printf("Error getting current working directory");
        return -1;
    }

    printf("Original Working Directory: %s\n", cwd);

    // Create a new directory
    if (mkdir(NEW_DIR, 0755) != 0) {
        printf("Error creating new directory");
        return -1;
    }

    // Change to the new directory
    if (chdir(NEW_DIR) != 0) {
        printf("Error changing to new directory");
        return -1;
    }

    // Print the new working directory
    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        printf("Error getting new working directory");
        return -1;
    }

    printf("New Working Directory: %s\n", cwd);
```

```
    // Change back to the original directory
    if (chdir("..") != 0) {
        printf("Error changing back to original directory");
        return -1;
    }

    // Remove the new directory
    if (rmdir(NEW_DIR) != 0) {
        printf("Error removing new directory");
        return -1;
    }

    printf("Removed directory '%s' and changed back to original directory.\n", NEW_DIR)

    return 0;
}
```

Video Reference:

https://youtube.com/playlist?list=PLWjmN065fOfGdAZrlP6316HVHh8jlve

4.4 Lab Exercises

Exercise 1: Create a C program that prompts the user to enter a directory name and uses the `mkdir` system call to create the directory.

Exercise 2: Write a program that opens the current directory using `opendir` and reads its contents using `readdir`, then displays the list of directory entries.

Exercise 3: Create a C program to delete a directory specified by the user using the `rmdir` system call.

Exercise 4: Write a program that uses the `getcwd` system call to retrieve the current working directory and displays it to the user.

4.5 Sample Viva Questions

Question 1: How is the mkdir system call used in C programming?

Question 2: Discuss the significance of the rmdir system call.

5.3 Sample Programs

1. A program to create a child process using fork system call.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t child_pid;

    // Create a child process
    child_pid = fork();

    if (child_pid == 0) {
        // The child process code section
        printf("Child process: PID = %d\n", getpid());
    } else if (child_pid > 0) {
        // The parent process code section
        printf("Parent process: Child PID = %d\n", child_pid);
    } else {
        // Fork failed
        printf("Fork failed\n");
        return 1;
    }

    return 0;
}
```

2. C program to demonstrates the creation of an orphan process.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    pid_t child_pid = fork();

    if (child_pid == 0) {
        // Child process
        printf("Child process: PID = %d\n", getpid());
        sleep(2); // Sleep to ensure the parent process terminates first
        printf("Child process: My parent's PID = %d\n", getppid());
    } else if (child_pid > 0) {
        // Parent process
        printf("Parent process: PID = %d\n", getpid());
        printf("Parent process: Terminating...\n");
```

```
    } else {
        printf("Fork failed\n");
        return 1;
    }

    return 0;
}
```

3. C program to demonstrate the creation of a Zombie process.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t child_pid = fork();

    if (child_pid == 0) {
        // Child process
        printf("Child process: PID = %d\n", getpid());
        exit(0); // Child process exits immediately
    } else if (child_pid > 0) {
        // Parent process
        printf("Parent process: PID = %d\n", getpid());
        printf("Parent process: Child PID = %d\n", child_pid);
        sleep(10); // Sleep to allow time for the child to become a zombie
        printf("Parent process: Terminating...\n");
    } else {
        printf("Fork failed\n");
        return 1;
    }

    return 0;
}
```

4. C Program to Demonstrate the Use of execv() System Call

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
```

```
    pid_t pid;  // Variable to hold the process ID
    char *args[] = {"ls", "-l", NULL};  // Arguments for the command

    // Create a new process
    pid = fork();

    if (pid < 0) {
        // If fork() fails, print an error message
        printf("fork failed");
        return -1;
    }

    if (pid == 0) {
        // This block is executed by the child process

        // Print a message to indicate the child process is running
        printf("Child process (PID: %d) running 'ls -l' command...\n", getpid());

        // Execute the 'ls -l' command
        execv("/bin/ls", args);

        // If execv() fails, print an error message
        printf("execv failed");
        exit(-1);  // Exit the child process with an error status
    } else {
        // This block is executed by the parent process

        // Wait for the child process to finish
        wait(NULL);

        // Print a message to indicate that the child process has finished
        printf("Parent process (PID: %d): Child process has finished.\n", getpid());
    }

    return 0;  // Return success
}
```

5. C Program to Demonstrate the Use of kill() System Call

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
int main() {
    pid_t pid;  // Variable to hold the process ID

    // Create a new process
    pid = fork();

    if (pid < 0) {
        // If fork() fails, print an error message
        printf("fork failed");
        return -1;
    }

    if (pid == 0) {
        // This block is executed by the child process

        // Print a message to indicate the child process is running
        printf("Child process (PID: %d) is running...\n", getpid());

        // The child process will wait indefinitely until it receives a signal
        while (1) {
            pause();  // Wait for signals
        }

        // This line will never be reached if the process is terminated by a signal
    } else {
        // This block is executed by the parent process

        // Print a message to indicate that the parent process is sending a signal
        printf("Parent process (%d) sending signal to child (%d)\n", getpid(), pid);

        // Send the SIGTERM signal to the child process
        kill(pid, SIGTERM);

        // Wait for the child process to terminate
        wait(NULL);

        // Print a message to indicate that the child process has been terminated
        printf("Parent process: Child process has been terminated.\n");
    }

    return 0;  // Return success
}
```

Video Reference:

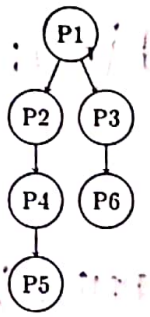
https://youtube.com/playlist?list=PLWjmN065fOfGdAZrlP6316HVIIh8jlve

5.4 Lab Exercises

Exercise 1: Write a C program to illustrate that performing 'n' consecutive fork() system calls generates a total of $2^n - 1$ child processes. The program should prompt the user to input the value of 'n'."

Exercise 2: Write a C program utilizing the `fork()` system call to generate the following process hierarchy: P1 → P2 → P3. The program should display the Process ID (PID) and Parent Process IDs (PPID) for each process created.

Exercise 3: Write a C program to generate a process hierarchy as follows:



The program should create the specified process structure using the appropriate sequence of 'fork()' system calls. Print PID and PPID of each process.

5.5 Sample Viva Questions

Question 1: Explain the fork system call in process management.

Question 2: Discuss the significance of the return values of the fork system call.

Question 3: What is the purpose of the wait system call?

Question 4: Discuss the significance of the getpid and getppid system calls in obtaining process IDs.

6 Experiment 6: Creation of Multithreaded Processes using Pthread Library

6.1 Objective

Introduce the operations on threads, which include initialization, creation, join and exit functions of thread using pthread library.

6.2 Reading Material[1][3][2]

6.2.1 Commonly used library functions related to POSIX threads (pthread)

Function	Description	Programming Syntax
pthread_create	Create a new thread	int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg);
pthread_join	Wait for termination of a specific thread	int pthread_join(pthread_t thread, void **retval);
pthread_exit	Terminate calling thread	void pthread_exit(void *retval);
pthread_cancel	Request cancellation of a thread	int pthread_cancel(pthread_t thread);

Table 5: Commonly used library functions related to POSIX threads (pthread)

6.3 Sample Programs

1. A C program using the pthread library to create a thread with NULL attributes.

```
#include <stdio.h>
#include <pthread.h>

void *thread_function(void *arg) {
    printf("Inside the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread_id;
    pthread_create(&thread_id, NULL, thread_function, NULL);
    pthread_join(thread_id, NULL);
    return 0;
}
```

2. A C program that creates a thread and passes a message from the main function to the thread.

```
#include <stdio.h>
#include <pthread.h>

void *thread_function(void *message) {
    printf("Message received in thread: %s\n", (char *)message);
    return NULL;
}

int main() {
    pthread_t thread_id;
    pthread_create(&thread_id, NULL, thread_function, "Hello from the main thread!");
    pthread_join(thread_id, NULL);
    return 0;
}
```

3. A C program where a thread returns a value to the main function using pointers.

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 1

void *thread_function(void *arg) {
    int *returnValue = malloc(sizeof(int));
    *returnValue = 143; // Set the return value
    pthread_exit(returnValue);
}

int main() {
    pthread_t threads[NUM_THREADS];
    int *thread_return;

    pthread_create(&threads[0], NULL, thread_function, NULL);
    pthread_join(threads[0], (void **)&thread_return);

    printf("Value returned from thread: %d\n", *thread_return);

    free(thread_return); // Free allocated memory for return value
    return 0;
}
```

7.2.4 Mutex

Definition	A mutex (Mutual Exclusion) is a synchronization primitive used in multi-threaded programming to control access to shared resources. It allows only one thread at a time to access the resource, preventing concurrent access.
Functionality	Mutexes ensure mutual exclusion, preventing race conditions and maintaining data integrity. Operations include initialization (`pthread_mutex_init`), locking (`pthread_mutex_lock`), unlocking (`pthread_mutex_unlock`), and destruction (`pthread_mutex_destroy`).
Types	Mutexes can be recursive (allows the same thread to lock it multiple times) or non-recursive (deadlocks if the same thread tries to lock it multiple times).

Function	Description	Programming Syntax
`pthread_mutex_init`	Initialize a mutex	`int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);`
`pthread_mutex_destroy`	Destroy a mutex	`int pthread_mutex_destroy(pthread_mutex_t *mutex);`
`pthread_mutex_lock`	Lock a mutex	`int pthread_mutex_lock(pthread_mutex_t *mutex);`
`pthread_mutex_unlock`	Unlock a mutex	`int pthread_mutex_unlock(pthread_mutex_t *mutex);`

Table 7: Library Functions Related to Mutex

7.3 Sample Programs

1. C Program Simulating Race Condition.

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define MAX_COUNT 1000000

int shared_variable = 0;

void *increment_variable(void *thread_id) {
    for (int i = 0; i < MAX_COUNT; i++) {
        shared_variable++;
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
```

```
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment_variable, (void *)i);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Value of shared variable after race condition: %d\n", shared_variable);

    return 0;
}
```

2. C Program with Semaphore to Avoid Race Condition.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define NUM_THREADS 5
#define MAX_COUNT 1000000

int shared_variable = 0;
sem_t semaphore;

void *increment_variable(void *thread_id) {
    for (int i = 0; i < MAX_COUNT; i++) {
        sem_wait(&semaphore);
        shared_variable++;
        sem_post(&semaphore);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    sem_init(&semaphore, 0, 1); // Initializing semaphore with value 1

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment_variable, (void *)i);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
```

```
    sem_destroy(&semaphore); // Destroying semaphore

    printf("Value of shared variable after synchronization: %d\n", shared_variable);

    return 0;
}
```

3. C Program with Mutex to Prevent Race Condition

```
#include <stdio.h>
#include <pthread.h>

#define NUMTHREADS 5
#define MAXCOUNT 1000000

int shared_variable = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *increment_variable(void *thread_id) {
    for (int i = 0; i < MAXCOUNT; i++) {
        pthread_mutex_lock(&mutex);
        shared_variable++;
        pthread_mutex_unlock(&mutex);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUMTHREADS];
    pthread_mutex_init(&mutex, NULL); // Initializing mutex

    for (int i = 0; i < NUMTHREADS; i++) {
        pthread_create(&threads[i], NULL, increment_variable, (void *)i);
    }

    for (int i = 0; i < NUMTHREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&mutex); // Destroying mutex

    printf("Value of shared variable after synchronization: %d\n", shared_variable);

    return 0;
}
```